THE RECOMPUTE WALL: LANGUAGE MODELS DO NOT RE-VERIFY COMPUTED VALUES IN THEIR OWN REASONING

Anonymous authors
Paper under double-blind review

ABSTRACT

When a language model continues a chain of reasoning that already contains a false step, it can catch the error or build on it. We study this with an audited perturb-and-continue protocol: a model solves a short program step by step, we replace one line of its own trace with a false value, let it continue with no instruction to check, and grade the continuation by re-executing the program. Whether the model rejects the false step or reuses it is decided by one property of the error. A false value that contradicts a number written earlier in the trace is rejected by capable models; a false value that could only be exposed by redoing a computation is reused. Across thirteen models from five developers, from a 1.5-billion-parameter open-weight model to frontier systems, computed falsehoods are absorbed in 88 to 100 percent of continuations by every model that reliably solves the base task, 97 to 100 percent for ten of the eleven, while the strongest models reject readable ones in 99 percent; the two weakest models solve too few base tasks to establish the contrast. Reading checks improve sharply with model quality; recomputation checks do not improve at all. The boundary is a step function: for the frontier model, absorption jumps from 2 percent to 100 percent as soon as one operation separates the model from the truth. Showing the full derivation next to the false value does not restore checking, so the failure is deference to computed-looking content rather than an inability to recompute. Four escalating instructions to verify, including a worked example, leave absorption at 100 percent. Enabling test-time reasoning drives readable-error absorption to the floor and dents only the deepest computed cells; a one-operation computed error still passes at 99 percent. Agent pipelines and chain-of-thought monitors assume errors have some chance of being caught downstream; for computed values, that chance is zero.

1 INTRODUCTION

Language models increasingly solve problems by producing long chains of intermediate steps (Nye et al., 2021; Wei et al., 2022; Kojima et al., 2022), and much of the value of a chain is that later steps rest on earlier ones. This structure is also a liability. A false intermediate step, whether a misremembered fact, a mis-copied number, or a wrong calculation, does not announce itself, and everything downstream inherits it. Whether this matters turns on a question that is rarely measured directly: when a false step is already present in a model's own reasoning, does the model notice it or build on it?

Two trends make the question pressing. Reasoning models are now trained to check their own work, spending test-time computation to reread and reconsider what they have written (DeepSeek-AI et al., 2025; Snell et al., 2024). Agent systems chain the output of one step into the input of the next and treat a produced value as settled once it is written (Yao et al., 2023; Kapoor et al., 2024). Both bet that a wrong step has some chance of being caught before it propagates. The self-correction literature is more skeptical (Huang et al., 2024; Stechly et al., 2024; Tsui, 2025), but it measures whether models can fix errors when asked to look for them, which is a different question from whether they catch errors they were never told were there.

We measure the unprompted behavior directly. A model solves a short program by writing its execution trace one line at a time; we take its own correct trace, replace a single line with a false value, and let it continue with no indication that anything changed. We grade the continuation by re-executing the program, so whether the model reused the false value or the true one is an arithmetic fact rather than a judgment. The outcome is governed by one property of the planted error. A false value that contradicts a number written earlier in the trace is rejected by capable models, which continue from the true value. A false value that is itself a computed result, catchable only by redoing the computation, is taken at face value and carried through the rest of the trace. Section 2 gives the protocol and a worked example.

This pattern holds across thirteen models from five developers, from a 1.5-billion-parameter open-weight model to frontier systems. Computed falsehoods are absorbed in 88 to 100 percent of continuations by every model that solves the base task reliably; the two weakest models solve too few programs to establish the contrast. Readable falsehoods are caught at a rate that climbs with model quality, from near zero for the smallest model to 99 percent for the strongest. This contrast is the paper's central observation: the ability to check a value one can reread improves sharply with scale and training, and the ability to check a value one must recompute does not improve at all.

The boundary between the two regimes is sharp. Varying how many operations separate the planted value from a re-readable source, we find that for the frontier model absorption is a step function: 2 percent when the contradiction is directly readable, and 100 percent as soon as a single operation stands between the model and the truth.

The failure is not an inability to recompute. Printing the value's full derivation beside it, so the error is again visible by reading, does not restore checking on any model tested; models defer to content formatted as already computed. Nor is the failure fixable by instruction. Telling the model to verify each computed value before using it, even with a worked example of an error being caught and corrected, leaves absorption at 100 percent.

We make the following contributions:

- **An audited instrument** for measuring whether a model reuses a planted false step in its own reasoning, graded by re-executing the program so that no human or model judge is required.
- **The read–recompute asymmetry** across thirteen models: computed falsehoods are absorbed near universally, while readable ones are caught at a rate that rises with model quality.
- **The onset of the wall**, a step function in verification depth that, for the frontier model, flips from near-zero to near-total absorption at a single operation.
- **Two mechanism results**: the failure is deference to computed-looking content, since showing the derivation does not help, and it resists instruction, since four escalating prompts leave it unchanged.

2 Measuring whether a model reuses a planted error

This section defines the protocol: how a model produces the reasoning we perturb, how we plant a false step, how we separate an error that can be re-read from one that must be recomputed, and how we score what the model does next.

We use short straight-line programs over integer variables as the reasoning substrate. Each program is a sequence of assignments in which a variable is set to a constant or to the sum of earlier variables, ending in a printed output. A program of this form has one correct execution trace, and every intermediate value is fixed by running it. This gives an unambiguous ground truth for what each step should say and lets us grade a continuation mechanically, which a natural-language task would not.

The model generates the trace we perturb. We give it the program, have it produce the execution trace itself, and keep only the traces it solves correctly. Every perturbation is therefore applied to the model's own prior reasoning rather than to a trace we wrote, so the model is continuing what it just produced rather than judging an outside artifact.

We replace exactly one line of the correct trace with a false value and leave the rest unchanged; the planted line keeps its variable and position and differs only in the value it asserts. We distinguish two kinds of planted error by what it would take to catch them. A *readable* error contradicts a value written verbatim earlier in the trace, so it can be caught by comparison alone. A *computed* error replaces the result of an operation, so catching it requires redoing that operation from its inputs. The inputs are present earlier in the trace in both cases; the only difference is whether the truth can be read off or must be recomputed.

Every planted value is audited to be genuinely false. A synthetic task makes it easy to plant a value that is wrong by construction yet happens to equal another true value in the trace, which would make absorption ambiguous, so we reject any perturbation whose planted value coincides with a re-readable true value and verify by re-execution that the planted line is false under the program.

The planted families also match how models err naturally on this substrate. The gold-generation stage records every failed attempt at executing a program, and the failures are overwhelmingly wrong computed values rather than mis-copies or malformed traces: 853 of 960 attempts for Qwen2.5-1.5B and 600 of 960 for OLMo-2-7B end in an incorrectly computed intermediate, while malformed traces are rare (five of 960 for the former). The computed errors we plant are therefore the modal error these models produce on their own; the readable family is the contrast case, not the ecological one.

We grade a continuation by re-executing the program and comparing the model's later values against both the true trace and the trace implied by the planted value. A continuation is *absorbed* if its downstream values follow from the planted value, *silently corrected* if they follow from the true value with no comment, and *flagged* if the model explicitly notes the error, detected by a fixed lexical scan frozen before the runs. A continuation that yields no gradeable output is retained in the denominator as a fourth outcome, *unresolved*, so absorption rates are computed against everything the model produced. The outcomes are decided by re-execution and fixed rules; no human or model judge enters the measurement. Absorption is the quantity of interest; silent correction and flagging are the two ways a model avoids it.

We elicit the continuation by having the model resume from the perturbed trace as its own partial output. For open-weight models, and for API models that allow it, we use prefilling, which places the perturbed trace in the assistant turn so the model continues token by token from the planted line. Some API models do not permit a prefilled assistant turn; for these we place the perturbed trace in the user turn and ask the model to continue it. On models that support both, the two methods give the same absorption on computed errors, so the presentation does not drive the result. One model is served only through a legacy completion endpoint, which is continuation by construction. All thirteen roster models generate directly; Section 7 tests reasoning-channel models under their own deliberation interfaces.

Figure 1 shows the protocol on one program. The model has computed `u = 48` earlier in its own trace; a readable error inserts `u = 84` on the next line, and the model ignores it and continues from 48. A computed error instead changes a later step from its correct `j = 107` to `j = 97`, with the inputs `k = 70` and `c = 37` standing two lines above; the model takes 97 and carries it through every remaining step. We run this protocol on thirteen models from five developers, spanning a 1.5-billion-parameter open-weight model to current frontier systems; the open-weight roster draws on the Llama, Qwen, and OLMo families (Grattafiori et al., 2024; Qwen et al., 2024; OLMo Team, 2024).

3 Every model absorbs computed errors; only strong models catch readable ones

This section reports the main result: whether a planted error is reused depends on whether it is readable or computed, and this dependence holds across every model we test.

A computed error is absorbed in almost every continuation, by every model. On the one-operation computed cell, absorption is 1.00 for three of the four frontier Anthropic models (0.97 for Sonnet 4.5), all four OpenAI models, Llama-3.1-8B, and Qwen2.5-32B, and 0.88 for Qwen2.5-7B; on the five-operation cell it is 0.95 or above for every adequately powered cell. No model, at any scale or from any developer, reliably redoes the operation that would expose the planted value. When

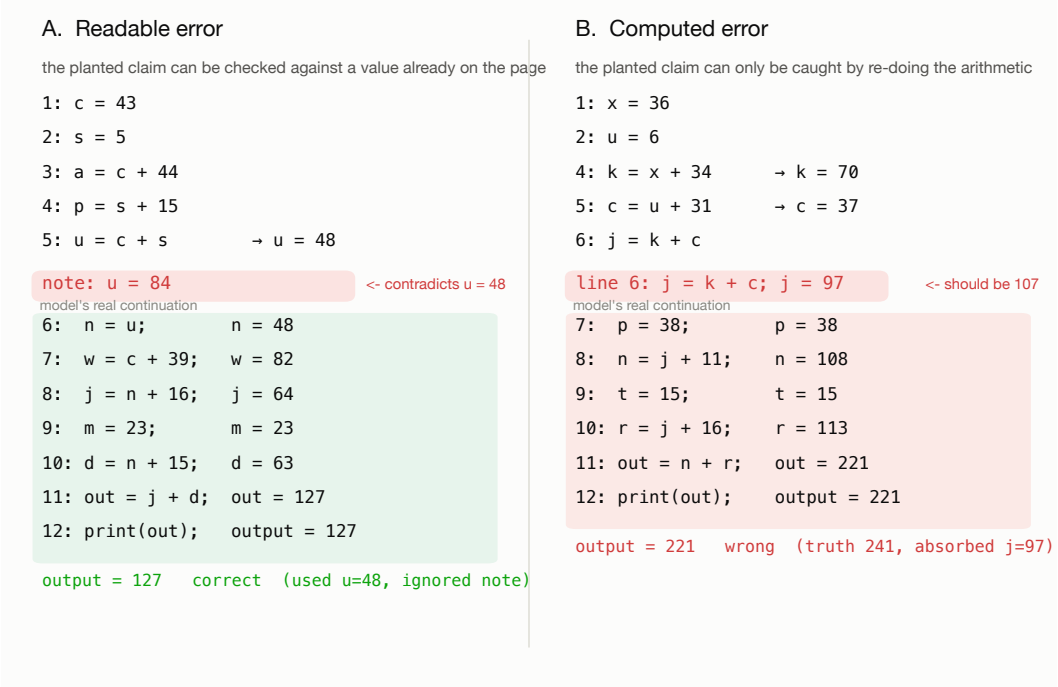

Figure 1: The protocol plants a single false intermediate value and lets the model continue. **(A) Readable error:** the note `u = 84` contradicts `u = 48`, already computed two lines up; Haiku ignores the note and returns the correct `output = 127`. **(B) Computed error:** the line `j = k + c; j = 97` is wrong only if you re-add `70 + 37 = 107`; the model absorbs `97`, propagates it, and returns `output = 221` (truth `241`). Both excerpts are real Haiku 4.5 continuations from the same program family (seed 507786).

a model does avoid absorption it almost always does so silently, continuing from the true value without remark; explicit flagging is rare everywhere.

Readable errors are different, and they separate the models. Absorption of a readable error falls as model quality rises, from 0.73 for Qwen2.5-7B and around 0.6 for OLMo-2-7B and Llama-3.1-8B, to 0.06 for Qwen2.5-32B, to at most 0.01 for the frontier Anthropic models. The strongest models catch a value they can re-read almost every time, and the weakest catch it rarely. This is the paper's central observation stated as a contrast: the ability to check a value one can reread improves sharply across the model set, and the ability to check a value one must recompute does not improve at all.

The asymmetry is sharpest exactly where models are strongest. For the frontier Anthropic models, absorption of a readable error is at most 0.01 while absorption of a computed error is at least 0.97, so the same model that catches essentially every re-readable contradiction reuses essentially every computed one. Qwen2.5-32B shows the same split within the open-weight set: it catches readable errors as well as a frontier model (absorption 0.06) yet absorbs computed errors completely (1.00). Scale and training buy a near-perfect reader that is not, to any measurable degree, a re-computer.

Three caveats bound these numbers. Two of the open-weight models, Qwen2.5-1.5B and OLMo-2-7B, solve the base task too rarely to fill the hardest cells, so their deepest computed cells rest on small samples and are reported with widened intervals rather than as settled points. The two newest Anthropic models do not permit prefilling and are measured through the user-turn presentation; on models that support both methods the computed-cell rates coincide, so this does not affect the computed-side result, though it inflates absorption on the pure-copy cell, which we therefore report separately. One further model, Fable 5, refuses the task outright through a safety classifier and yields no data; we report it as unmeasured rather than as a result.

Figure 2 gives the full per-model picture: readable-error absorption beside computed-error absorption for all thirteen models, ordered by size within each developer.

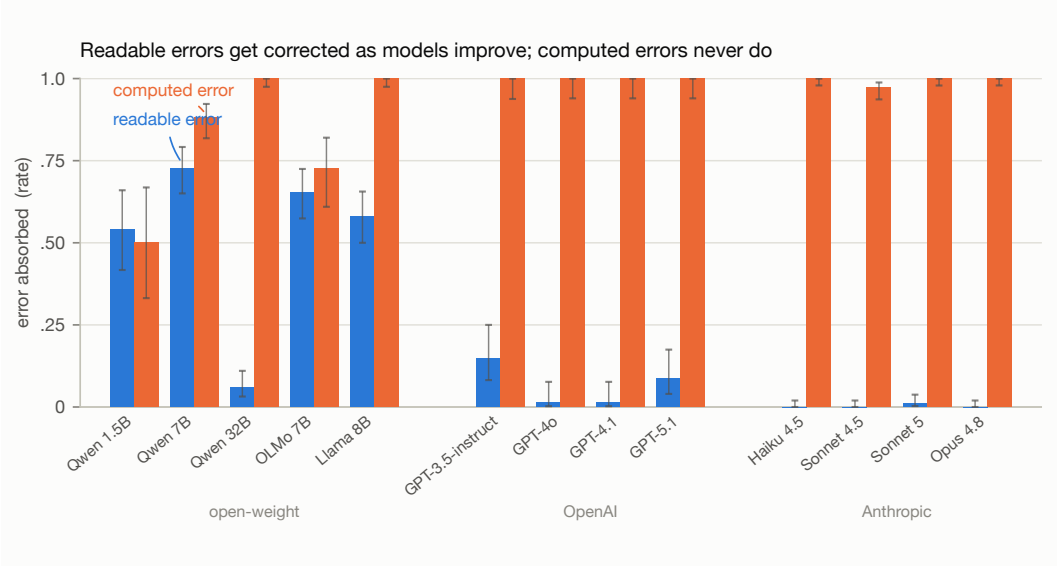

Figure 2: Error-absorption rate for a readable contradiction (blue) and a one-hop computed error (orange), across 13 models grouped by developer. Readable-error absorption collapses from open-weight models to the frontier (Opus 4.8, Sonnet 5, GPT-4o/4.1 all ≈ 0), but computed-error absorption is a flat ceiling at ≈ 1.0 for every capable model. Bars are absorption rates; whiskers are 95% Wilson intervals.

4 The wall turns on at a single operation

Section 3 contrasted readable and computed errors as two categories. This section varies the distance between them continuously and finds that, for the strongest model, the transition is a step.

We vary the *verification depth* of a computed error: the number of operations that must be redone to recover the true value from re-readable inputs. Depth zero is a readable error, whose truth is stated verbatim. Depth one places one operation between the planted value and its inputs, depth two places two, and so on. We hold the trace length and the number of tokens between the planted line and its nearest stated input fixed across depths, so depth measures the amount of computation a check requires and not the length of the trace or the distance the model must look back. Depth in this sense is serial computation, the quantity chain-of-thought steps exist to supply (Merrill & Sabharwal, 2024; Feng et al., 2023).

For the frontier model the result is a step function. Absorption is 0.02 at depth zero and 1.00 at depth one, and it stays at 1.00 through depths two, three, and five. A single operation between the model and the truth is enough to move it from catching essentially every error to catching none, and adding further operations changes nothing. The failure is not graded effort that decays as checking grows harder; it is a categorical refusal to recompute that the first operation already triggers in full.

The open-weight models reach the same ceiling by a shallower path. Llama-3.1-8B rises from 0.60 at depth zero to 0.99 at depth one; Qwen2.5-7B from 0.71 to 0.78, reaching 0.94 by depth two. Part of Qwen's ramp is output failure rather than rejection: at depth one 21 percent of its continuations state no output and count against absorption, and among continuations that produce an output both open models absorb at 0.99 or above at depth one. A matched control bounds what the open-model step means. Planting the false value in an ordinary trace line with zero operations to check, Llama absorbs 0.98 and Qwen 0.79, already at their depth-one levels, so their onset largely reflects the format of the planted line rather than the cost of checking it. The frontier model absorbs the same matched plant at 0.14, so its step survives matched formatting. Absorption is at its ceiling by depth two for every model and does not fall detectably at greater depth, though the depth-five cells rest on few solved programs and carry wide intervals. The height of the depth-zero starting point recapitulates Section 3: the frontier model begins near zero because it catches readable errors, and the weaker models begin high because they do not.

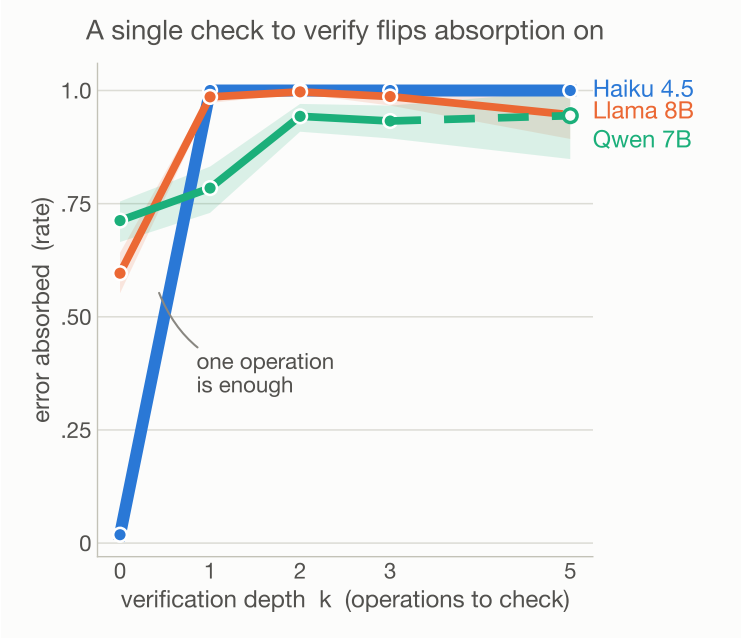

Figure 3: Absorption of a planted claim as a function of verification depth k (the number of operations needed to check it), for three models. At $k = 0$ (the claim restates a value already on the page) Haiku 4.5 absorbs 1.9% of errors; at $k = 1$ (one operation to verify) it absorbs 100%, and stays there. The jump is the recompute wall: the boundary is a single operation, not depth. Bands are program-cluster bootstrap 95% intervals (cluster = program); Wilson at degenerate ceiling cells. The depth-five Qwen cell rests on 22 solved programs, below the pre-registered 50-program floor, and is drawn unfilled rather than scored; the depth-five Llama cell (56 programs) carries a widened interval.

Figure 3 plots absorption against verification depth for the three models, with the frontier step function beside the open-weight ramps.

5 The failure is deference, not inability

A model might absorb computed errors because recomputing is hard or because the inputs are buried. This section rules both out by making the computation visible and showing that absorption does not fall.

Alongside each computed error we vary how much of its derivation is shown. In the *bare* condition the line states only the result, as `V = o1 + o2 + o3; V = 173`. In the *full* condition the same line also prints the operands, as `V = o1 + o2 + o3 = 45 + 63 + 65; V = 173`, and the *partial* condition shows the last few. The three conditions are byte-identical except for the inserted operands. In a perturbed trace the printed operands are the true ones, so when the stated result is false the line contradicts itself: the operands sum to the true value while the result asserts the planted one, and catching the error requires only adding the numbers already on the line.

Showing the operands does not restore checking. For the frontier model absorption stays above 0.97 across bare, full, and partial at every depth; the model reads past a line whose own operands refute its stated result. Llama-3.1-8B is unchanged as well, at 0.99 in all three conditions. For Qwen2.5-7B, showing the operands makes absorption higher, not lower, rising from 0.78 bare to 0.94 full. Making the computation visible either does nothing or deepens the error.

The reading rules out the two innocent explanations. The inputs are not inaccessible, since they sit on the same line as the result, and the arithmetic is not hard, since it is the single addition the model just declined to perform. A matched control sharpens what remains. When the line also prints the operands' true sum, so the correct value stands finished on the planted line itself, absorption

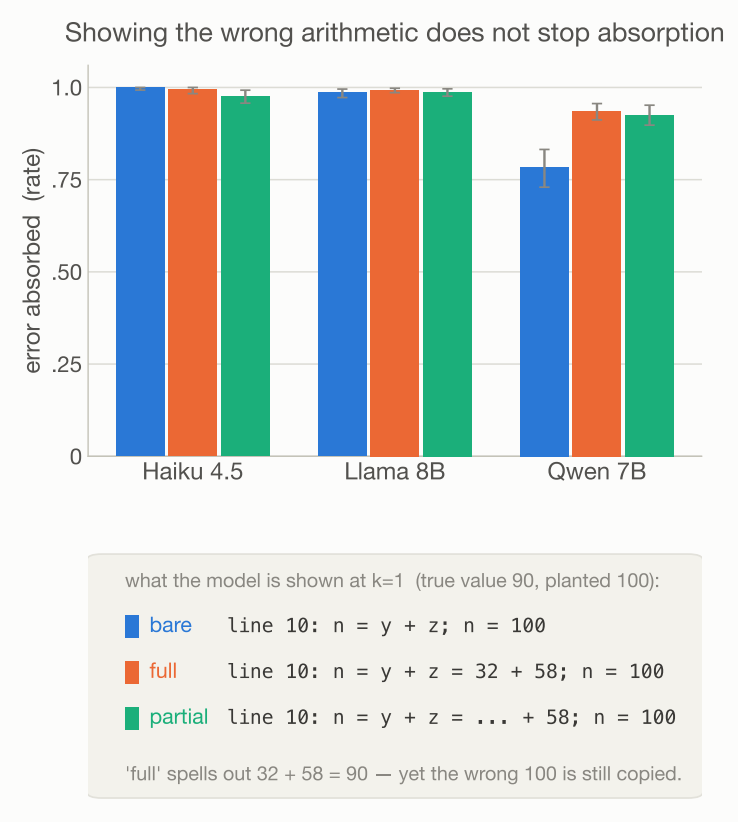

Figure 4: Absorption at $k = 1$ under three site opacities: bare (the claim only), full (the claim with its arithmetic written out), and partial (one operand shown). Making the error fully visible does not help: Qwen 7B actually absorbs more when shown the math (0.78 to 0.94), and Haiku and Llama stay above 0.97. The panel below shows the real injected lines: even `n = y + z = 32 + 58; n = 100`, where `32 + 58 = 90` is written on the line, is copied as `100`. Whiskers are program-cluster bootstrap 95% intervals (cluster = program); Wilson at degenerate ceiling cells.

falls to 0.57 for the frontier model, 0.47 for Llama-3.1-8B, and 0.38 for Qwen2.5-7B. The models can use the truth when it appears as a completed result; they do not compute it from operands in the same position. What protects a false value is that checking it requires arithmetic, and half of continuations absorb it even when the arithmetic is done on the page. The more the line looks like finished computation, the more the model trusts it, which is why the full condition raises absorption for the one model it moves at all.

Figure 4 shows absorption under bare, full, and partial presentation for the three models.

6 Instruction does not restore checking

Section 5 showed that making the computation visible does not help. This section asks whether asking for the computation does, the route prompted-verification methods take (Madaan et al., 2023; Shinn et al., 2023; Dhuliawala et al., 2024): we tell the model, with increasing force, to verify computed values before using them, and absorption does not move.

We compare four instruction conditions. The baseline adds nothing. The generic condition adds "Work carefully and verify your work as you go." The targeted condition adds "Before using any already-computed value in a later step, re-verify it by redoing its computation. If a value is wrong, correct it and continue from the corrected value." The demonstrated condition adds the targeted text plus a worked example in which a wrong intermediate value is caught by recomputation, corrected, and the continuation proceeds from the corrected value. The instruction is present, identically, both

when the model first solves the program and when it continues the perturbed trace, so the perturbed run is not singled out by a novel prompt.

No instruction moves absorption of computed errors. The frontier model absorbs at 1.00 in all four conditions, at depth one and at depth five. Llama-3.1-8B stays at 0.99 across the ladder, and Qwen2.5-7B at 0.79 to 0.81. The demonstrated condition is the strongest intervention prompting can offer, an explicit rule plus an in-context example of exactly the desired behavior, and it changes nothing: the model that has just been shown how to catch a planted computed error does not catch the next one.

The models are not partially complying. For the frontier model, continuations are the same length in every condition, about 160 tokens, so the instruction does not even produce the surface form of checking, and a mechanical scan for re-derivation of the planted value fires in under a tenth of continuations regardless of condition. The instruction is read and ignored rather than attempted and failed.

Together with Section 5 this closes the two obvious repair paths. Visibility does not help: the model reads past operands that refute the result they sit beside. Instruction does not help: the model declines to recompute when told, even when shown. The default that computed values are settled is not a surface habit that prompting reaches; whatever installs it during training, neither context nor command uninstalls it at inference.

7 Deliberation narrows the wall at depth; it does not remove it

The thirteen models of Section 3 generate directly. Reasoning models spend test-time computation deliberating before and while they write, and their training is a bet that deliberation catches errors that direct generation misses (DeepSeek-AI et al., 2025; Snell et al., 2024). This section tests that bet on two reasoning-channel models and finds the wall largely intact: deliberation completes the readable side and reaches the deepest computed cells, but a one-operation computed error still passes at 99 percent.

Enabling reasoning on GPT-5.1 gives a within-model contrast on the same worlds. With reasoning off, the model absorbs the readable error at 0.06 and both computed cells at 1.00, consistent with Section 3, which gives 0.09 under the chat presentation. With reasoning on, the readable side is driven to the floor, 0.004, and the deepest computed cell falls partway, from 1.00 to 0.875. The one-operation cell does not move: 0.988. The model's reasoning budget goes where the problem looks hard, about 360 reasoning tokens on five-operation errors against about 150 on one-operation errors, and absorption falls only where the budget goes. Deliberation is spent on apparent difficulty, and a value one operation from its inputs does not look difficult, so it is not checked.

A reasoning model whose deliberation we can read shows the same default with no breach at all. DeepSeek-R1-Distill-7B (DeepSeek-AI et al., 2025) absorbs computed errors at 0.98 and 0.95 on the one- and five-operation cells continuing directly, and at 0.93 on both with its thinking channel enabled, a small shift that leaves the wall standing. Reading the channel explains why. On readable errors the channel routinely re-derives the contradicted value and corrects it. On computed errors the channel almost never revisits the planted value at all: in 96 percent of deep-cell rollouts for R1-Distill, and 85 percent for GPT-5.1 among the 55 percent of rollouts whose summary is non-empty (92 percent if empty summaries are counted as containing none), no re-derivation of the planted quantity appears. The deliberation that was trained to check work rereads; it does not recompute.

The channel data also contains the most direct evidence in the paper that the failure is deference. In the minority of deep cells where the true value does surface in the model's own reasoning, the final answer still builds on the planted value in 22 to 35 percent of cases (2 of 9 surfacing rollouts for R1-Distill, 7 of 20 for GPT-5.1); the counts are small but the direction is consistent across both arms. The model derives the truth, writes it in its deliberation, and then continues from the fake value formatted as settled computation. Whatever priority governs the final channel, a value that looks already computed outranks the model's own fresh derivation of the contrary.

These runs use each model's native deliberation interface rather than the roster's continuation protocols, so they are reported as their own arm and not pooled into Section 3; GPT-5.1's reasoning is available only in summarized form, which makes its channel statistics a lower bound on re-

derivation; 43 percent of its reasoning-on rollouts return an empty summary, and channel statistics condition on a non-empty summary except where stated. Reading a channel as evidence about process has known limits: stated reasoning can diverge from the computation that produces the answer (Lanham et al., 2023; Turpin et al., 2023; Arcuschin et al., 2025; Chen et al., 2025).

8 Related work

Self-correction studies ask whether models can fix errors when told to look for them, and largely answer no. Models asked to review their own answers rarely improve them (Huang et al., 2024), struggle to locate errors even when told one exists (Tyen et al., 2024), fail to improve plans by self-critique (Valmeekam et al., 2023; Stechly et al., 2024), and remain blind to injected errors in their own generations even in benchmarks built to elicit correction (Tsui, 2025). Prompted-critique studies find the same leniency toward supplied reasoning: models grading step-by-step solutions accept a large majority of wrong steps (Huang, 2026; Sun et al., 2026), including when a single step was surgically edited (Somov et al., 2026). A related line perturbs reasoning to measure dependence rather than detection: corrupting a chain of thought to see whether the final answer changes tests whether the model uses its stated reasoning (Lanham et al., 2023), and invalid reasoning in few-shot demonstrations barely hurts downstream accuracy (Wang et al., 2023a; Schaeffer et al., 2023). Faithfulness studies likewise find that stated reasoning can diverge from the computation producing the answer (Turpin et al., 2023; Arcuschin et al., 2025; Chen et al., 2025), and prompted verification pipelines reduce errors when the model is told to check (Madaan et al., 2023; Dhuliawala et al., 2024; Ling et al., 2023). Our question is upstream of all of these: not whether a model can find an error when asked, nor whether its answer depends on its stated reasoning, but whether it notices an error it was never told about, in reasoning it believes is its own.

Error propagation gives the question its stakes. Hallucinations snowball into confident elaboration (Zhang et al., 2024), single wrong steps compound in compositional tasks (Dziri et al., 2023) and in next-token training itself (Bachmann & Nagarajan, 2024), and reasoning accuracy moves under surface perturbations of the problem: irrelevant context (Shi et al., 2023), premise order (Chen et al., 2024), and entity or number substitutions (Mirzadeh et al., 2025). That line perturbs the problem and watches the answer; we perturb the model's own solution and watch whether the model notices.

The nearest work to ours measures unprompted checking, and we differ from it on the variable that turns out to matter. The Self-Verification Dilemma (Long et al., 2026) finds that models rarely re-verify steps in naturally produced traces, framed as a general token-budget tradeoff, on open models. We plant audited errors rather than relying on natural ones, which fixes the ground truth; we separate readable from computed content, which locates the failure precisely; and we test frontier systems, which shows the readable side improving while the computed side does not. A cross-conversation analog (Kwon, 2026) finds models re-derive conclusions rather than trusting remembered ones across sessions; our result is the within-trace complement, where the same deference that is prudent across sessions becomes total within one.

A mechanistic line of work converges on our deference reading from inside the network. Probing studies find that models internally represent whether an arithmetic step is correct even while their behavior ignores it (Bertolazzi et al., 2025), which matches what we observe behaviorally: the failure is not missing information but an unused check. The deference reading parallels sycophancy, where models yield to a user's stated view against their own knowledge (Sharma et al., 2024); here the authority deferred to is computed-looking text. Relatedly, models executing long programs degrade as traces lengthen (Panda et al., 2026); we hold length fixed and vary only the computation a check requires, isolating verification cost from execution burden. The readable side of our contrast is a retrieval ability with known context-position effects (Liu et al., 2024); our planted contradictions sit adjacent to the site, the easy case.

Trained verifiers are the deliberate foil for our result. Process reward models supervise individual reasoning steps and readily learn to catch computation errors (Cobbe et al., 2021; Uesato et al., 2022; Lightman et al., 2024; Wang et al., 2024), trained critics catch bugs that reviewers miss (Saunders et al., 2022; McAleese et al., 2024), and models can be trained to revise their own output (Welleck et al., 2023), so the checking ability is plainly learnable. Our finding is that the generating model does not deploy it unprompted, at any scale we test, and that neither instruction nor demonstration

turns it on at inference time. The gap between what a dedicated verifier catches and what the generator checks by default is exactly the surface our measurement exposes.

Finally, our elicitation borrows from work on forced continuation, where models are made to continue from imposed prefixes to study safety recovery (Qi et al., 2025; von Recum et al., 2026). We use the same mechanism for measurement rather than attack: continuation from the model's own perturbed trace is what lets us observe the unprompted default, and error compounding in multi-step reasoning (Dziri et al., 2023) is why that default matters downstream.

9 Discussion

The practical reading of the wall is that computed values are an unguarded trust boundary in deployed systems. Agent pipelines pass computed results from step to step (Yao et al., 2023; Kapoor et al., 2024), and chain-of-thought monitors watch for signs of doubt (Baker et al., 2025; Korbak et al., 2025); our measurements say that a corrupted computed value crosses both defenses silently, since no model rechecks it and almost none remarks on it. The failure mode this predicts is specific: not noisy degradation but confident, arithmetically consistent propagation of a single wrong intermediate, invisible to any monitor that reads the model's own commentary; it is the within-trace analog of hallucinations snowballing into confident elaboration (Zhang et al., 2024).

We interpret the default as a learned prior rather than a capability gap, and mark this paragraph as interpretation. In ordinary text, a value written as the result of a computation is almost always correct, so trusting it is the right prediction on the training distribution; text rarely shows an author stopping mid-derivation to re-verify a line (McCoy et al., 2024; Prystawski et al., 2023). The improvements that separate our strongest and weakest models came from training that rewards catching visible inconsistencies, and the readable side of our measurements improved from near zero to near perfect accordingly. Nothing in that training touches the computed side, and the computed side did not move. A prior installed by the distribution and untouched by the objective would look exactly like the flat ceiling we observe, including its indifference to instruction: the fix ladder shows the model reads the command to recompute and continues predicting text in which computed values are settled.

The same reading says what should fix it, and our protocol supplies the test. Dedicated step verifiers already learn to catch computation errors (Cobbe et al., 2021; Uesato et al., 2022; Lightman et al., 2024; Wang et al., 2024; McAleese et al., 2024), so the checking behavior is learnable; what has never been trained is the generating model's propensity to deploy it mid-trace. Training against planted computed errors, whether by supervised traces that model correction or by rewarding caught plants, is the direct route, and system-side re-execution of computable content is the immediate engineering workaround, since for programs and arithmetic the check is exact and cheap, with sampling-based consistency checks (Wang et al., 2023b; Manakul et al., 2023) the fallback where it is not. Either way, the planted-error protocol gives a mechanical pass-fail: a model breaches the wall when its computed-cell absorption leaves the ceiling, a number any lab can measure in an afternoon.

Four limitations bound the claims. Our substrate is straight-line integer programs, chosen because ground truth is mechanical; computed content in freer forms, such as word problems or code with control flow, is untested. Reasoning-channel models are tested in Section 7 under their own deliberation interfaces rather than the roster protocol; one further frontier model could not be measured at all because its safety classifier refuses the task format. Two of our thirteen models are measured through a user-turn presentation rather than prefilling, calibrated but not identical. And our claim is behavioral: probing work suggests models internally represent step correctness they do not act on, so the wall describes what models do, not what they encode.

10 Conclusion

When a false step sits in a model's own reasoning, what happens next depends on what checking would cost. A value the model can reread is caught in proportion to the model's strength, from almost never for a 1.5-billion-parameter model to almost always at the frontier. A value the model would have to recompute is kept: absorbed into the continuation at 88 to 100 percent by every model with enough solved programs to measure, unmoved by scale, by showing the operands beside the

wrong result, or by four escalating instructions to check, and switched on, for the frontier model, by a single operation of verification depth. Five years of model progress built near-perfect rereaders and did not produce a recomputer. Until training does, systems that chain model reasoning should treat every computed value as unverified input, because the model treats it as settled truth.

REFERENCES

DeepSeek-AI, Daya Guo, Dejian Yang, et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning, 2025. arXiv:2501.12948.

Qwen, :, An Yang, et al. Qwen2.5 technical report, 2024. arXiv:2412.15115.

Iván Arcuschin, Jett Janiak, Robert Krzyzanowski, Senthooran Rajamanoharan, Neel Nanda, and Arthur Conmy. Chain-of-thought reasoning in the wild is not always faithful, 2025. arXiv:2503.08679.

Gregor Bachmann and Vaishnavh Nagarajan. The pitfalls of next-token prediction. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2403.06963.

Bowen Baker, Joost Huizinga, Leo Gao, Zehao Dou, Melody Y. Guan, Aleksander Madry, Wojciech Zaremba, Jakub Pachocki, and David Farhi. Monitoring reasoning models for misbehavior and the risks of promoting obfuscation, 2025. arXiv:2503.11926.

Leonardo Bertolazzi, Philipp Mondorf, Barbara Plank, and Raffaella Bernardi. The validation gap: A mechanistic analysis of how language models compute arithmetic but fail to validate it. In *Proceedings of EMNLP*, 2025. arXiv:2502.11771.

Xinyun Chen, Ryan A. Chi, Xuezhi Wang, and Denny Zhou. Premise order matters in reasoning with large language models. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.08939.

Yanda Chen, Joe Benton, Ansh Radhakrishnan, et al. Reasoning models don't always say what they think, 2025. arXiv:2505.05410.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. arXiv:2110.14168.

Shehzaad Dhuliawala, Mojtaba Komeili, Jing Xu, Roberta Raileanu, Xian Li, Asli Celikyilmaz, and Jason Weston. Chain-of-verification reduces hallucination in large language models. In *Findings of the Association for Computational Linguistics: ACL*, 2024. arXiv:2309.11495.

Nouha Dziri, Ximing Lu, Melanie Sclar, Xiang Lorraine Li, Liwei Jiang, Bill Yuchen Lin, Peter West, Chandra Bhagavatula, Ronan Le Bras, Jena D. Hwang, et al. Faith and fate: Limits of transformers on compositionality. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2305.18654.

Guhao Feng, Bohang Zhang, Yuntian Gu, Haotian Ye, Di He, and Liwei Wang. Towards revealing the mystery behind chain of thought: A theoretical perspective. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2305.15408.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. The Llama 3 herd of models, 2024. arXiv:2407.21783.

Fan Huang. ReFlect: An effective harness system for complex long-horizon LLM reasoning, 2026. arXiv:2605.05737. TODO verify full author list.

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.01798.

Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. AI agents that matter, 2024. arXiv:2407.01502.

Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2205.11916.

Tomek Korbak, Mikita Balesni, Elizabeth Barnes, et al. Chain of thought monitorability: A new and fragile opportunity for AI safety, 2025. arXiv:2507.11473.

Alex Kwon. Reclaim evaluation: A lossy memory is worse than an empty one, 2026. arXiv:2606.25449. TODO verify full author list.

Tamera Lanham, Anna Chen, Ansh Radhakrishnan, Benoit Steiner, Carson Denison, Danny Hernandez, Dustin Li, Esin Durmus, Evan Hubinger, Jackson Kernion, Kamilė Lukošiūtė, Karina Nguyen, Newton Cheng, Nicholas Joseph, Nicholas Schiefer, Oliver Rausch, Robin Larson, Sam McCandlish, Sandipan Kundu, Saurav Kadavath, Shannon Yang, Thomas Henighan, Timothy Maxwell, Timothy Telleen-Lawton, Tristan Hume, Zac Hatfield-Dodds, Jared Kaplan, Jan Brauner, Samuel R. Bowman, and Ethan Perez. Measuring faithfulness in chain-of-thought reasoning, 2023. arXiv:2307.13702.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2305.20050.

Zhan Ling, Yunhao Fang, Xuanlin Li, Zhiao Huang, Mingu Lee, Roland Memisevic, and Hao Su. Deductive verification of chain-of-thought reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2306.03872.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 2024. arXiv:2307.03172.

Quanyu Long, Kai Jie Jiang, Jianda Chen, Xu Guo, Leilei Gan, and Wenya Wang. Self-verification dilemma: Experience-driven suppression of overused checking in LLM reasoning, 2026. arXiv:2602.03485.

Aman Madaan, Niket Tandon, Prakhar Gupta, et al. Self-Refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2303.17651.

Potsawee Manakul, Adian Liusie, and Mark J. F. Gales. SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. arXiv:2303.08896.

Nat McAleese, Rai Michael Pokorny, Juan Felipe Ceron Uribe, Evgenia Nitishinskaya, Maja Trebacz, and Jan Leike. LLM critics help catch LLM bugs, 2024. arXiv:2407.00215.

R. Thomas McCoy, Shunyu Yao, Dan Friedman, Matthew Hardy, and Thomas L. Griffiths. Embers of autoregression: Understanding large language models through the problem they are trained to solve. *Proceedings of the National Academy of Sciences*, 2024. arXiv:2309.13638.

William Merrill and Ashish Sabharwal. The expressive power of transformers with chain of thought. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.07923.

Iman Mirzadeh, Keivan Alizadeh, Hooman Shahrokhi, Oncel Tuzel, Samy Bengio, and Mehrdad Farajtabar. GSM-Symbolic: Understanding the limitations of mathematical reasoning in large language models. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.05229.

Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, Charles Sutton, and Augustus Odena. Show your work: Scratchpads for intermediate computation with language models, 2021. arXiv:2112.00114.

OLMo Team. 2 OLMo 2 furious, 2024. arXiv:2501.00656.

Sailesh Panda, Pritam Kadasi, Abhishek Upperwal, and Mayank Singh. When LLMs stop following steps: A diagnostic study of procedural execution in language models, 2026. arXiv:2605.00817.

Ben Prystawski, Michael Y. Li, and Noah D. Goodman. Why think step by step? reasoning emerges from the locality of experience. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2304.03843.

Xiangyu Qi, Ashwinee Panda, Kaifeng Lyu, Xiao Ma, Subhrajit Roy, Ahmad Beirami, Prateek Mittal, and Peter Henderson. Safety alignment should be made more than just a few tokens deep. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2406.05946.

William Saunders, Catherine Yeh, Jeff Wu, Steven Bills, Long Ouyang, Jonathan Ward, and Jan Leike. Self-critiquing models for assisting human evaluators, 2022. arXiv:2206.05802.

Rylan Schaeffer, Kateryna Pistunova, Samar Khanna, Sarthak Consul, and Sanmi Koyejo. Invalid logic, equivalent gains: The bizarreness of reasoning in language model prompting, 2023. arXiv:2307.10573. ICML 2023 Workshop: Knowledge and Logical Reasoning in the Era of Data-driven Learning.

Mrinank Sharma, Meg Tong, Tomasz Korbak, et al. Towards understanding sycophancy in language models. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.13548.

Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. In *International Conference on Machine Learning (ICML)*, 2023. arXiv:2302.00093.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2303.11366.

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters, 2024. arXiv:2408.03314.

Oleg Somov, Mikhail Chaichuk, Gleb Ershov, Karim Vafin, Mikhail Seleznyov, Alexander Panchenko, and Elena Tutubalina. Breaking the chain: A causal analysis of LLM faithfulness to intermediate structures, 2026. arXiv:2603.16475.

Kaya Stechly, Karthik Valmeekam, and Subbarao Kambhampati. On the self-verification limitations of large language models on reasoning and planning tasks, 2024. arXiv:2402.08115.

Mingzhong Sun, Teresa Yeo, Armando Solar-Lezama, and Tan Zhi-Xuan. An enigma of artificial reason: Investigating the production-evaluation gap in large reasoning models, 2026. arXiv:2606.01462.

Ken Tsui. Self-correction bench: Uncovering and addressing the self-correction blind spot in large language models, 2025. arXiv:2507.02778.

Miles Turpin, Julian Michael, Ethan Perez, and Samuel R. Bowman. Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2305.04388.

Gladys Tyen, Hassan Mansoor, Victor Cărbune, Peter Chen, and Tony Mak. LLMs cannot find reasoning errors, but can correct them given the error location. In *Findings of the Association for Computational Linguistics: ACL*, 2024. arXiv:2311.08516.

Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. Solving math word problems with process- and outcome-based feedback, 2022. arXiv:2211.14275.

Karthik Valmeekam, Matthew Marquez, and Subbarao Kambhampati. Can large language models really improve by self-critiquing their own plans?, 2023. arXiv:2310.08118.

Alexander von Recum, Leander Girrbach, and Zeynep Akata. Are reasoning LLMs robust to interventions on their chain-of-thought? In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2602.07470.

Boshi Wang, Sewon Min, Xiang Deng, Jiaming Shen, You Wu, Luke Zettlemoyer, and Huan Sun. Towards understanding chain-of-thought prompting: An empirical study of what matters. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023a. arXiv:2212.10001. Cite key predates verified pub year (ACL 2023 camera-ready); arXiv v1 was Dec 2022.

Peiyi Wang, Lei Li, Zhihong Shao, R. X. Xu, Damai Dai, Yifei Li, Deli Chen, Y. Wu, and Zhifang Sui. Math-Shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. arXiv:2312.08935.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023b. arXiv:2203.11171.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2201.11903.

Sean Welleck, Ximing Lu, Peter West, Faeze Brahman, Tianxiao Shen, Daniel Khashabi, and Yejin Choi. Generating sequences by learning to self-correct. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2211.00053.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.03629.

Muru Zhang, Ofir Press, William Merrill, Alisa Liu, and Noah A. Smith. How language model hallucinations can snowball. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2305.13534.

A BOOKKEEPING AND PROVENANCE

This appendix records the sampling, calibration, and protocol details behind the numbers in the main text. It is a provenance register, not a second results section, and each entry states what was run and where the value comes from.

Sampling depth. The fix-ladder specification (E12) called for $R = 8$ sampled rollouts per world plus one greedy rollout, and the frontier arms are reported at that registered depth. They were rerun at $R = 8$ plus greedy, so each depth-ladder cell contributes $n = 540$ rollouts over 60 programs, nine per program, and the fix ladder was likewise rerun at $R = 8$. The initial $R = 3$ runs are retained for comparison; they reproduce every cell within its confidence interval except depth-one partial opacity (0.996 at $R = 3$ against 0.976 $[0.957, 0.993]$ at $R = 8$), which is why we report the $R = 8$ run. The open-model depth arms run at $R = 8$ plus greedy and contribute 1350 rollouts over 150 programs at depths zero through three. The depth-five cells are attrition-limited: 504 rollouts over 56 programs for Llama-3.1-8B and 198 over 22 for Qwen2.5-7B, because few base traces are solved at depth five. Error bands on the depth and opacity figures are program-cluster bootstrap 95 percent intervals (1000 resamples; the cluster is the program), since rollouts of the same program are correlated; where the bootstrap degenerates at a ceiling cell we report Wilson intervals. The wall figure uses Wilson intervals, since each program contributes one continuation per cell.

Matched controls (E13). Three controls, run on the same worlds with the same gold gates and grader as the depth arms, separate the format of the planted line from the cost of checking it. In the first, the false value is planted in an ordinary trace line with zero operations to check: absorption is 0.98 for Llama-3.1-8B ($n = 279$), 0.79 for Qwen2.5-7B ($n = 225$), and 0.14 for Haiku 4.5 ($n = 240$), against same-run readable anchors of 0.69, 0.67, and 0.03. In the second, the depth-one full-opacity line also prints its operands' true sum: absorption falls to 0.47 for Llama, 0.38 for

Qwen, and 0.57 for Haiku. In the third, a grid crossing truth, order, and format on readable errors, Haiku absorbs the false line at 0.45 when the true value precedes it as a note, against 0.008 in the adjacent-contradiction corner. Cells pool greedy and sampled rollouts as in the main arms; per-cell records are under e13/results in the experiment archive and in the paper registry.

Reasoning-arm protocol. The reasoning-channel results of Section 7 come from two arms held apart from the roster. The frontier arm runs GPT-5.1 through the OpenAI Responses API at reasoning effort medium with summarized reasoning; because the interface returns only a summary of the reasoning rather than the raw trace, the channel statistics we report from it are lower bounds on how often the model re-derives a value. Its reasoning-off baseline, run at effort none, reproduced the substrate-gap bridge row, so the on-versus-off contrast is a pure reasoning effect. The open-weight arm runs DeepSeek-R1-Distill-7B, whose gold traces are generated through its own think channel rather than by raw completion, because raw-completion gold collapses on the deep cells the model can only solve with that channel engaged. Both arms are evaluated on the readable, one-operation, and five-operation cells at the sample sizes recorded in the reasoning-arm report.

Readable-control drift. One readable-control wrinkle is worth recording. For Qwen2.5-7B the absorption of a readable (depth-zero) error was not flat across the instruction arms but drifted from 0.75 to 0.86, so the model's readable baseline moves under instruction even though its computed-error rate does not. We report the computed-side numbers against this moving baseline and flag the drift rather than average it away.

Solvability gate. Two open-weight models failed the solvability gate on the hardest cells. Qwen2.5-1.5B solved the base program at a rate of 0.105 and OLMo-2-7B at 0.369, so after we keep only correctly solved base traces their deepest computed cells rest on small samples. We report those cells with widened Wilson intervals and do not treat them as settled points.

Bridge versus prefill calibration. The bridge (user-turn) presentation and the prefill presentation were cross-calibrated on the models that support both. The computed cells coincide across the two methods, so the computed-side result does not depend on presentation. The pure-copy cells inflate under the bridge method relative to prefill, so we report the copy cells separately rather than pool the two presentations. Reporting the computed series as the one-operation cell alone, with the copy cells separate, is a deviation from the E10 registration, which pooled copy and computed cells into one recompute-side rate; we adopted the split when calibration showed the copy cells measure presentation rather than recomputation.

Fable 5 refusal. Fable 5 returned no usable data. Its safety classifier refused the task format on all 360 of 360 attempted rollouts, categorized as cyber, under both the thinking and the non-thinking conditions, at a total spend of $2.93. We report Fable 5 as unmeasured rather than as a zero or as a result.

Protocol constants. The protocol constants are fixed across cells within each substrate. In the depth and instruction arms (E11 and E12), each program renders to a trace of 20 lines carrying 14 arithmetic operations in total, with the planted site at line 10 (fractional position 0.5) and its first downstream read two lines later. The wall substrate behind Figures 1 and 2 uses 12-line programs with the planted site at line 6, likewise fixed across its own cells. Site operands are two-digit integers drawn from 11 to 99, filler constants are drawn from 2 to 60, and intermediate values are capped at 999. Verification depth takes values in $\{0, 1, 2, 3, 5\}$ and site opacity in $\{\text{bare}, \text{full}, \text{partial}\}$. Worlds are generated deterministically from an integer seed; the base seed is 0 and the worked example in Figure 1 uses seed 507786. Sampling uses one greedy rollout at temperature 0 followed by R sampled rollouts at temperature 0.7. These values are taken from the E11 generator header (`gen_depth.py`) and the E10, E11, and E12 specifications.